Assignment, WASP Software Engineering Course Module 2026

Felix Loyau-Kahn

August 31, 2026

1 Introduction

My research focuses on static analysis, particularly Reference Attributed Grammars (RAGs). RAGs offer a declarative framework for compiler construction in which attributes are associated with nodes in the Abstract Syntax Tree (AST). By expressing static analyses and semantic behavior through these attributes, RAGs enable the development of compilers that are modular, extensible, and maintainable.

This thesis explores novel strategies for systematically and verifiably identifying the causes of false-positive and false-negative bug reports in static checkers. The research focuses on the generation and use of contestable traces to improve the precision, refinement, and configurability of static analyses. A contestable trace is a structure that represents an explanation of a result made by a static analysis, designed so that each step can be inspected, verified, refined, and contested to identify sources of imprecision.

The work will be carried out within the declarative meta-compilation framework JastAdd and the ExtendJ Java compiler. Different aspects of contestable trace generation and analysis refinement will be explored through the implementation of software artifacts, experimental evaluation, and publication of both scientific results and tools, with a focus on practicality, automation, and scalability.

2 Lecture principles

2.1 Design By Contract

In the course, Design By Contract is defined as: “If all the routine’s preconditions are met by the caller, the routine shall guarantee that all postconditions and invariants will be true when it completes.”

You then state that if the language doesn’t support it, we should approximate with assertions. I think one of the goals of a compiler based on RAGs is to fulfill that requirement at a language definition level, providing a framework to express semantic rules and static analyses, acting like contracts which rely on assumptions, preventing runtime bugs by catching them at compilation time, verifying that contracts are respected before execution. That follows the “Crash early” and “Crash, don’t trash”.

2.2 Verification and Validation

V&V also relates with my work: verification is central to static analysis, as it strongly participates in checking whether we are building the product right, and validation is more indirectly related, because better explanation can help developers determine good specification, checking whether we are building the right product.

By defining semantic rules that a language should follow, and static analysis that verifies the implementation of software to ensure they follow the language specification. This belongs to the first of the three groups you state: static techniques, dynamic techniques, and process evaluation techniques.

3 Guest-lecture principles

3.1 Requirements Engineering

Requirements Engineering (RE) is related to my work: we specify semantic rules that a language must fulfil, and RE defines a prescriptive statement of intent that describes an abstract property (requirements specification) that a system must fulfill.

RE specifies what a software should do, while semantic rules specify what valid programs are allowed to do. In both cases, it's the same motivation: the cost of defect removal; the later a defect is detected, the more it will cost to remove it. It is better to catch it in an early phase or define it as a requirement from the start.

3.2 Behavioral Software Engineering

As defined in the slides: BSE is "applying psychological and social principles to address issues affecting software development."

This is of interest in my research, developers make predictable mistakes, static analysis is designed to compensate for these human mistakes like division by zero, null pointer dereferences, and other developer mistakes. We want to ensure that many people can work together on a project while ensuring the global quality of the produced implementation stays high, preventing bugs from going into production, which could be a potential risk. My work of making contestable traces makes warnings easier to understand, with better explanations and influences developer behaviour.

4 Data scientists, software engineers, and AI engineers

Data scientists come from a scientific background and mainly does statistics and machine-learning algorithms, their main activities are gathering and cleaning data and developing models, which is a model-centric point of view. Software engineers focus on delivering software products and has a system-wide point of view to meet the users and clients requirements, designing the architecture of a system, implementing, testing, and deploying it, analyzing risks, and designing safety and security mechanisms into the system, bringing it to deployment in production-like expectations.

This idea of a system-wide view and model-centric view applies well to my research area of compilation and static analysis: ExtendJ is a large software system, and my research

may introduce AI-assisted analyses, which need to integrate into deterministic compiler infrastructure, therefore requiring system-wide perspective, with the AI part being considered as black-box functions that could return incorrect results. Our domain of expertise is far from that of data scientists, being more centered on producing a well-structured codebase. If AI-assisted static analyses become part of future compiler, developers will need explanation mechanisms to verify the AI-generated results before trusting them in production, which should be part of my contestable traces work.

After reading these chapters, I think data scientists and software engineers will remain distinct roles. Their skills have little overlap, and they need to work together to build production ML systems. Specialization remains valuable in my opinion to keep both system-wide view and model-centric view consistent and well produced.

But it will become more important that both have some knowledge of the other's domain: from the data scientists perspective, to meet the needs, requirements, and hardware constraints of the global project; and from the software engineer's perspective, to incorporate the models accordingly into a large-scale project that can be deployed in production with a good level of safety and security.

Some new concepts and technologies tend to change the boundaries between these roles: foundation models reduce the amount of task-specific model development, AI coding assistants reduce the barrier for software engineers to use ML, Retrieval-Augmented Generation (RAG) systems used for domain-specific tasks like either data or software engineering, and finally, prompt pipelines with agentic workflows can assist any role and code entire applications with minimal human intervention, ML-enabled projects included.

The term "AI Engineering" highlights a stronger focus on engineering in machine-learning projects. It mostly refers to building automated pipelines, deploying models, MLOps, and it tends to promote system-wide rather than model-centric points of view, which is becoming increasingly necessary. It is at the center of these two chapters.

5 Paper analysis

5.1 First paper: An Exploratory Study of V-Model in Building ML-Enabled Software: A Systems Engineering Perspective

The first paper I chose is written by Jie JW Wu, published in the CAIN 2024 conference: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI.

5.1.1 Core ideas and their SE importance

The paper continues the discussion from the first two chapters of "Machine Learning in Production", they also observe the challenges of the development process for ML-enabled software in large-scale real-world applications due to data quality, technical debt, testing, security & privacy, safety, and interdisciplinary collaborations.

They apply a systems engineering lens to investigate the use of the V-Model to address these challenges, and by establishing several propositions to manage interdisciplinary collaborations when building products with ML components.

This paper fits very well into the subject of engineering of AI systems and takes a closer look at the application of a specific software engineering methodology to find its pros and cons. It opens up future research directions such as system decomposition, clear system boundaries, and consistency of Validation & Verification (V&V) for building ML-enabled systems.

5.1.2 Relation to my research

In my research, while working on a declarative compiler with the aim of semantic verification and static analysis, we try to keep our software as deterministic as possible. The subject of building AI-assisted RAG-based declarative compilers is still an emerging research area.

We are going to try and use AI components in very specific parts of our compiler. In my research project, for example, to find counterexamples that explain false positives and negatives in some static analysis, which is normally NP-hard, but could potentially be solved more efficiently using well-trained AI. The solutions returned by the AI could easily be verified in linear complexity.

The integration of such ML components would have to be well isolated from the rest of the compiler, and since no ML component has been introduced yet, we still don't have any workflow on how to incorporate it into our software while keeping good isolation guarantees. This V-model could be an interesting solution on how to work with ML components for our future work.

5.1.3 Integration into a larger AI-intensive project

Let's imagine a software project that reads and classifies mail and bank checks automatically for a given list of clients. As a start, we build software that reads through some PDF and text files and does some pattern matching or fuzzy matching with a list of clients to sort each mail to the corresponding client. But then we want to introduce the classification of scanned mail, and to do so we have to perform optical character recognition (OCR) on the PDF using AI. Also, if we want to scan, for example, bank checks to automatically put all the information in a spreadsheet, which contains handwriting, we would also need AI to extract this handwritten text. So some AI components have to be introduced.

The difficulties introduced by adding ML components will mainly come from the collaboration of software and ML engineers, following their modified V-model lifecycle would help this collaboration by securing the process and enabling production-ready software.

5.2 Second paper: Less Is More: Measuring How LLM Involvement affects Chatbot Accuracy in Static Analysis

The second paper I chose was written by Krishna Narasimhan in 2026. It addresses several subjects: LLMs for software engineering, AI-assisted development, LLM/RAG/prompt evaluation, and Agentic software engineering.

5.2.1 Core ideas and their SE importance

This paper studies how LLMs should interact with a static analysis framework to answer code-analysis questions. The authors first introduce LLM involvement as a design variable and compare three architectures along a spectrum of LLM autonomy. The first translates a

natural language description of a code analysis task to CPGQL queries directly, the second transforms natural language to an intermediate representation in a JSON table and then passes that table through a deterministic algorithm to produce the static analyzer queries, and finally, the third gives full control to the LLM by letting it invoke the tool and pass the arguments itself. We have three levels of LLM involvement, which are compared in terms of correctness on known examples, large-scale projects, and LLM token usage.

5.2.2 Relation to my research

This paper is closely related to my research. We want to introduce LLMs for very specific tasks in our compiler, but we want to keep as much determinism as possible. This paper introduces three levels of LLM involvement in static analysis and shows that they are not trained on enough static-nalysis data to produce a DSL with correct semantics, and that it is a better choice to produce structured output which is then passed to deterministic algorithms to maximize correctness and optimize token usage.

5.2.3 Integration into a larger AI-intensive project

In our compiler, we have developed a lot of static analysis for Java, and we believe one of the main reasons that prevent people from using it is the difficulty of tracing false positives and negatives. We have in mind to try to use LLMs to produce concrete examples that show these errors to the user. This is an algorithmically complex problem, but one that could be verified in linear time after being produced by an LLM.

Since my own WASP research focuses on tracing these false positives, we could also use LLMs to identify which widening operation introduced the imprecision causing these false positives and negatives, and compare them with the result of a deterministic algorithm.

5.2.4 Adaptation of your research

The use of LLMs in a compiler is still uncertain, this paper has introduced ways to differentiate and compare LLM usage in three different ways. When we introduce AI into our project, we will be able to test different solutions, compare them, and choose accordingly depending on our needs, also comparing them with our own deterministic solution.

6 Evidence, originality, and GenAI-use transparency

- ”An Exploratory Study of V-Model in Building ML-Enabled Software: A Systems Engineering Perspective” is a peer-reviewed paper published in the CAIN conference proceedings. `https://dl.acm.org/doi/10.1145/3644815.3644951`
- ”Less Is More: Measuring How LLM Involvement affects Chatbot Accuracy in Static Analysis” is an arXiv paper released in 2026, not yet published in a peer-reviewed journal or conference proceedings. `https://arxiv.org/abs/2604.21746`
- ”Machine Learning in Production: From Models to Products” is a book that has been published by MIT Press as open access. `https://mlip-cmu.github.io/book/`
- The course material.

- The sources I trust the most are:
  - the first paper because it is peer-reviewed.
  - the second paper because although it is a preprint, it provides experimental evidence supporting its conclusions.
  - the MIT Press book is also highly trustworthy because it is an academic textbook.
- I used GenAI for language checking, after I finished the assignment.

7 Conclusion

This course was relevant for my future work which will probably include AI. It gave me insight into the current practices in AI engineering and how to use it safely and methodically to benefit fully from its capabilities while keeping software reliable and safe. It gave me a methodology for integrating AI components in large-scale projects, while keeping in mind the production constraints, discussing the many difficulties encountered by people working with AI in large-scale projects involving interdisciplinary teams.

For my research, after doing this course, I think AI should assist deterministic compiler analyses rather than replace them.

The course also changed the way I think about AI usage. Before the course, I viewed AI as an isolated component, whereas I now see it more as a part of a larger engineering problem to incorporate with a system-wide view. I really appreciated the importance of verification and production concerns.